Milestone 4 Group Report

Team: TFC
Tianhao Cao, Yusen Huang, Marco Wang, Darwin Zhang

1 Data

1.1 High-Level Description

The `UBC-SLIME/colx585_group_project_data` dataset is designed and a continuation of our Membership Inference Attacks (MIA), but with the addition of Vision-Language Models. Each of the samples we have viewed on `milestone4/document/data_inspection.md` consists of five features: a unique `id`, an `image` (stored as raw bytes), the target `is_member` binary label (0 or 1), a `text` string containing a conversational prompt and response, and a `type` category (e.g., `seen_img_unseen_txt`). The `type` feature is particularly interesting and useful as it seems to categorize if the image, text, or both has been seen during training which may be very useful.

1.2 Descriptive Statistics

Note: The text length calculations represent the character count of the combined prompt and response string.

Split	Samples	Mean Length	Median Length	Max	Min	Std Dev
train	6,000	261.93	246	813	98	89.24
validation	1,200	258.05	241	737	104	86.61
test	6,000	247.19	232	1,079	82	91.35

1.3 Literature Review

Paper 1: Membership Inference Attacks against Large Vision-Language Models

This paper addresses the important data security concerns regarding VLLMs' training process. The inclusion of sensitive private information such as private photos or medical records can be a very worrying issue. And how to detect such data is still an unresolved issue because of lack of datasets and right methodologies. In the paper, they introduced a benchmark to facilitate training data detection. They also included a novel MIA pipeline designed for token-level image detection and a whole new metric.

Short summary: **Main Task**: Detecting whether a specific image or text was used to train a vision-language model (i.e., membership inference attack on VLLMs). This is framed as a binary classification problem—given a data point, determine if it's a member or non-member of the training set.

Dataset: The authors constructed **VL-MIA**, which includes three sub-datasets: VL-MIA/DALL-E (592 member images from LAION-CCS vs. DALL-E-generated non-member images), VL-MIA/Flickr (600 MS COCO member images vs. recent Flickr photos as non-members), and VL-MIA/Text (600 samples of instruction-tuning text vs. GPT-4-generated answers as non-members). They also used the existing **WikiMIA** benchmark for LLM pre-training text detection.

Models: Three open-source VLLMs—**LLaVA 1.5**, **MiniGPT-4**, and **LLaMA-Adapter V2**. They also tested on closed-source **GPT-4** (vision-preview API).

Main Contribution: They released the first MIA benchmark (VL-MIA) specifically designed for VLLMs and they also proposed a **cross-modal pipeline** that enables image MIA by computing metrics from text logit slices, solving the problem that image tokens aren't directly accessible in VLLMs. They finally introduced **MaxRenyi-K%**, a target-free metric based on Renyi entropy that generalizes existing methods like Min-K% and works on both image and text modalities.

Citation: Li, Z., Wu, Y., Chen, Y., Tonin, F., Abad Rocamora, E., & Cevher, V. (2024). Membership Inference Attacks against Large Vision-Language Models. Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Paper 2: The Sample Complexity of Membership Inference and Privacy Auditing

This paper studies membership inference attacks (MIAs) from a theoretical perspective by asking how much auxiliary information an attacker needs in order to succeed. Instead of focusing directly on LLMs or VLMs, it studies a simpler setting called Gaussian mean estimation, where a model learns from n samples and releases a noisy estimate. The main result shows that an attacker may need far more reference samples than the model used for training, which differs from many existing MIA methods that only use $O(n)$ samples. The paper also argues that current privacy audits may underestimate risk if they do not fully use information about the data distribution.

Main Task. The paper studies the sample complexity of membership inference attacks. More specifically, it asks how many auxiliary samples from the same population are needed for an attacker to distinguish whether a target example was part of the training set or not. The analysis focuses on sample-based MIAs in Gaussian mean estimation and compares the unknown-covariance and known-covariance settings.

Dataset. This paper does not use a real benchmark dataset such as Flickr, LAION, or WikiMIA. Instead, it considers a theoretical setting where the data come from a d-dimensional Gaussian distribution $N(\mu, \Sigma)$, and the training set consists of n i.i.d. samples drawn from that distribution.

Models. The model studied in this paper is not a VLM or LLM, but a Gaussian mean estimator. The main estimator is the noisy empirical mean, which outputs

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} X_i + \rho Z, \quad Z \sim N(0, \Sigma).$$

The attacker then tries to infer whether a target sample was used in training based on this released estimate and a set of auxiliary samples.

Main Contribution. The paper shows that when the covariance is unknown, a successful sample-based MIA may require much more auxiliary data than expected, up to $\Omega(n + n^2\rho^2)$ samples. This means the attacker may need far more data than the model used for training. The paper also shows that the problem becomes much easier when the covariance is known, highlighting that estimating covariance structure is the main challenge. Overall, the paper suggests that many current MIA methods, which typically use only $O(n)$ reference samples, may underestimate real privacy risk.

Citation: Haghifam, M., Smith, A., & Ullman, J. (2025). The Sample Complexity of Membership Inference and Privacy Auditing. arXiv preprint arXiv:2508.19458.

2 Baseline Implementation Documentation

2.1 Overview

The primary goal of the script is to differentiate whether a given data sample belongs to the model's training set (member) or not (non-member) by calculating the model's loss on the sample. It loads a fine-tuned VLM model, extracts loss features for all data samples, and trains a Logistic Regression classifier on these features to output membership predictions on the test set.

2.2 Core Dependencies and Setup

- **Model**: `UBC-SLIME/colx_585_vlm` (built on the `SmolVLMForConditionalGeneration` architecture, utilizing SDPA attention for acceleration).
- **Dataset**: `UBC-SLIME/colx585_group_project_data`.
- **Compute Device**: Automatically detects CUDA availability. If available, it uses the GPU with `bfloat16` precision; otherwise, it falls back to the CPU with `float32` precision.

2.3 Core Modules Breakdown

2.3.1 Sample Loss Computation (`compute_loss`)

This is the core feature extraction function of the baseline method:

- **Image Processing**: Extracts image bytes from the sample and converts them to an RGB `PIL.Image`. It has fallback mechanisms using Base64 decoding or generating a dummy blank image if the image data format is problematic.
- **Text Processing**: Splits the dataset text into User instructions and Assistant responses. It then formats these using the model processor's `apply_chat_template` into a conversational format.
- **Calculation Mechanism**: Within a `torch.no_grad()` context, the forward pass calculates the cross-entropy loss of predicting the `assistant` text. This loss serves as the sole signal for inferring whether the sample is a training member.

2.3.2 Data Processing and Feature Caching

To minimize the significant overhead of repeated inference, the script incorporates a caching mechanism for three data splits, saving the extracted features into JSON files:

1. **Train Split (∼6000 samples)**:
   - Extracts `loss` and sets the `is_member` label to `1`.
   - Results are cached in `train_features.json`.
2. **Validation Split (∼1200 samples)**:
   - Extracts `loss` alongside the target `is_member` for performance evaluation.
   - Results are cached in `val_features.json`.
3. **Test Split (∼6000 samples)**:
   - Extracts only the `loss` for final inference.
   - Results are cached in `test_features.json`.

   Note: The script supports a `--debug` argument. When enabled, it processes a maximum of 100 samples per split and skips all caching (both reading and writing), which is useful for quick debugging.

2.3.3 Model Training and Evaluation

- **Logistic Regression Training**: Based on the `loss` extracted from the `train_features`, the script trains an `sklearn` Logistic Regression model. It uses the `class_weight='balanced'` parameter to automatically handle potential positive/negative class imbalances.
- **Metrics Evaluation**: The model's capabilities are evaluated on the Validation split, outputting two key attack metrics:
  - **ROC AUC Score**: Measures the model's overall ability to correctly rank and classify samples.
  - **TPR@FPR=0.1**: The False Positive Rate is held at 10%, and the True Positive Rate is reported (i.e., the proportion of actual members successfully identified).

2.3.4 Final Prediction and Output

After training, the classifier is applied to the test set (`test_features.json`).

- **Score Generation**: The prediction probabilities `predict_proba()[:, 1]` from the Logistic Regression model are used as continuous scores indicating the sample's likelihood of being a member.
- **Output Format**: The parsed `id` and the calculated `is_member` probabilities are outputted to the `milestone4/output/baseline_submission.csv` file, which serves as the final submission format for evaluation.

2.4 Equipment and Configuration

We are using a windows laptop with RTX 4070 Laptop GPU, 8GB VRAM, and 32GB RAM, with i9-13900HX CPU to run the script (15 minutes to run on my laptop). Additionally, we have access to silicon devices with 16GB.

3 Contributions

3.1 Milestone 4

Member	Contributions	Percentage
Tianhao Cao	Methods	25%
Yusen Huang	Methods	25%
Marco Wang	Methods	25%
Darwin Zhang	Literature and Data	25%

Total: 100%